

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

Verilog Assignment 5

August 2026

***Instructions:** This is a group assignment. Each group must implement the Verilog design, write a suitable testbench, program the FPGA board, and demonstrate the working design on the board to a Teaching Assistant for evaluation. The behavioral simulation must also be demonstrated to the Teaching Assistant.*

Question 1: Bidirectional Circular LED Shift Register

Design and implement a **5-bit bidirectional circular shift register** whose output is displayed using five LEDs on the FPGA board.

The circuit must operate using a **slow clock** derived from the on-board clock. The slow-clock generation method demonstrated in the laboratory using a counter and one of its higher-order bits may be used.

1. Top-Level Module Interface

Create a top-level module named

`bidirectional_led_shift`

with the following interface:

Signal	Width	Description
<code>clk</code>	1 bit	Input board clock
<code>reset</code>	1 bit	Reset input
<code>enable</code>	1 bit	Enables or freezes the LED movement
<code>direction</code>	1 bit	Selects left or right movement
<code>led</code>	5 bits	Output connected to five board LEDs

The board switches or push buttons may be used for `reset`, `enable`, and `direction`.

2. Slow Clock Generation

The FPGA board clock is too fast for the LED changes to be observed directly.

Use the clock-divider technique demonstrated in the laboratory.

A sufficiently wide counter may be incremented on every rising edge of the board clock:

`clk` $\longrightarrow$ `Clock-Divider Counter` $\longrightarrow$ `slow_clk`

A higher-order bit of the counter may be used as the slow clock. The counter width and selected bit should be chosen such that the LED movement is clearly visible to a human observer.

For example, the demonstrated approach used a counter of the form

`reg [25:0] clk_div`

and a higher-order bit of this counter as the slow clock.

Students may use the same idea or select another suitable higher-order bit depending on the FPGA board clock frequency.

Slow-Clock Board Demonstration

Before demonstrating the complete shift-register operation, students should be able to demonstrate that the slow clock has been generated correctly.

For example, the slow-clock signal may temporarily be connected to an LED so that the LED visibly turns ON and OFF.

Thus,

`Board Clock` $\longrightarrow$ `Clock Divider` $\longrightarrow$ `Slow Clock` $\longrightarrow$ `Blinking LED`

may be used as a simple demonstration of the generated slow clock.

3. Shift Register Operation

The circuit must contain a 5-bit register representing the five LEDs.

After reset, the register must contain

`00001`

so that exactly one LED is ON.

The register must update on every rising edge of the generated `slow_clk`.

4. Left Circular Shift

When

`enable = 1`

and

`direction = 0,`

the active LED must move towards the higher-order bit positions.

The required sequence is

`00001` $\rightarrow$ `00010` $\rightarrow$ `00100` $\rightarrow$ `01000` $\rightarrow$ `10000` $\rightarrow$ `00001` $\rightarrow \dots$

The bit shifted out from the left side must therefore re-enter from the right side.

5. Right Circular Shift

When

$$\text{enable} = 1$$

and

$$\text{direction} = 1,$$

the active LED must move towards the lower-order bit positions.

For example, starting from 00001, the sequence is

$$00001 \rightarrow 10000 \rightarrow 01000 \rightarrow 00100 \rightarrow 00010 \rightarrow 00001 \rightarrow \dots$$

Thus, the bit shifted out from one end must re-enter from the opposite end.

6. Enable Operation

The **enable** input controls whether the shift register is allowed to change.

enable	Required Operation
0	Hold the current LED pattern
1	Perform the selected circular shift

For example, if

$$\text{led} = 00100$$

and **enable** becomes 0, the LED pattern must remain

$$00100$$

irrespective of further slow-clock edges.

When **enable** becomes 1 again, shifting must continue from the currently stored pattern.

7. Direction Control

The direction input must have the following meaning:

direction	Operation
0	Circular shift towards led [4]
1	Circular shift towards led [0]

The direction must be changeable while the circuit is running.

For example, if the current state is

$$00100,$$

then:

$$\text{direction} = 0 \Rightarrow 01000$$

on the next slow-clock edge, whereas

$$\text{direction} = 1 \Rightarrow 00010.$$

8. Reset Operation

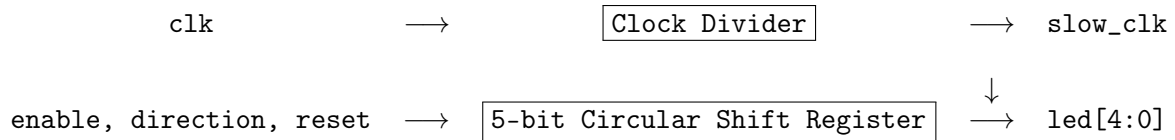
When `reset` is asserted, the LED pattern must return to

`00001`.

Therefore, after reset, the first LED must be ON and the remaining four LEDs must be OFF.

9. Overall Design Organization

The complete circuit can be viewed as two sequential blocks:



The clock-divider counter is updated using the board clock.

The LED shift register is updated using the generated slow clock.

10. Expected Board Operation

The following operations must be demonstrated on the FPGA board.

1. Reset

Assert reset and show that

`led = 00001`.

2. Left movement

Set

`enable = 1,      direction = 0`

and demonstrate the sequence

`00001 → 00010 → 00100 → 01000 → 10000 → 00001`.

3. Right movement

Change

`direction = 1`

and demonstrate that the active LED moves in the opposite direction.

4. Enable/Hold

Set

`enable = 0`

and demonstrate that the currently active LED remains fixed.

Set

`enable = 1`

again and demonstrate that the sequence resumes.

5. Slow-clock visibility

Students must demonstrate that the clock has been slowed sufficiently for the movement or blinking of the LEDs to be clearly visible on the board.

11. Testbench and Behavioral Simulation

Write a testbench for the sequential logic of the `bidirectional_led_shift` module.

The testbench must demonstrate:

- reset operation;
- left circular shifting;
- right circular shifting;
- enable and hold behavior; and
- change of direction during operation.

The behavioral simulation waveform must show at least

`clk, reset, enable, direction, led[4:0].`

Students may use a smaller clock-divider counter in the testbench so that the slow-clock transitions can be observed without requiring an excessively long simulation.

12. Verilog Design Requirements

The design must use sequential Verilog constructs appropriately.

Students must use:

- edge-triggered `always` blocks for sequential logic;
- non-blocking assignments (`<=`) for register updates;
- a register/counter for slow-clock generation; and
- a 5-bit register for storing the LED pattern.

The LED pattern must be generated by the sequential shift-register operation and must not be implemented as a combinational lookup table.

13. Mandatory FPGA Demonstration

Demonstration on the FPGA board is mandatory.

Each group must program the FPGA board and demonstrate the complete working circuit to a Teaching Assistant.

The demonstration must include:

- visible LED movement using the slow clock;
- left circular shifting;

- right circular shifting;
- enable/hold functionality;
- reset functionality; and
- correct response to the board switches or push buttons used as control inputs.

The Teaching Assistant may ask students to change the direction, enable/disable the movement, or reset the circuit during the live demonstration.

14. Submission

Submit the Verilog design source file(s), constraint file, and testbench in a single zipped folder named

`VerilogAssignment5_{Group_Number}.zip`.

The submitted files must include the group number, names and roll numbers of all group members, assignment number, and date of submission.